COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010-Data Structures and Algorithm	**SECTION:**	CPE21S1
DATE PERFORMED:	28/07/2026	**DATE SUBMITTED:**	28/072026
NAME:	Alvarez, Joshua Caleb P.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. _4.1_ Stacks

1. PROCEDURE OUTPUT

CODE

ILO A: Create a stack using the C++ STL

main.cpp

STACK > stack.cpp > main()

```cpp
//Alvarez, Joshua Caleb P.
#include <iostream>
#include <stack>

using namespace std;

int main(){
    //create an object for the stac
    stack<int> newStack;
    std:: cout << " Stack Empty?:  "<<newStack.empty()<<std::endl;
    //push to the stack
    newStack.push(3);
    newStack.push(8);
    newStack.push(15);

    std::cout<<"Stack Empty?:  "<<newStack.empty()<<std::endl;
    std::cout<<"The size of the stack: "<<newStack.size()<<std::endl;
    std::cout<<"The top of the element is: "<<newStack.top()<<std::endl;
    std::cout<<"Removing the elements from the stack: \n";

    std::cout<<"The top of the element: is: "<<newStack.top()<<std::endl;
    newStack.pop();
     std::cout<<"The top of the element: is: "<<newStack.top()<<std::endl;
    newStack.pop();
     std::cout<<"The top of the element: is: "<<newStack.top()<<std::endl;
    newStack.pop();

    return 0;
}
```

size.cpp

STACK > Stack2p0.cpp > size.cpp > ...

```cpp
// Alvarez, Joshua Caleb P.
#include <iostream>

const size_t maxCap = 100;
int stack[maxCap];      // Stack with max of 100 elements
int top = -1, i, newData;

void push();
void pop();
void Top();
bool isEmpty();
bool isFull();
void print();

int main() {

    int choice;

    std::cout << "Enter the number of max elements for the new stack: ";
    std::cin >> i;

    while (true) {

        std::cout << "\n===== STACK OPERATIONS =====\n";
        std::cout << "1. PUSH\n";
        std::cout << "2. POP\n";
        std::cout << "3. TOP\n";
        std::cout << "4. Is Empty\n";
        std::cout << "5. Print the elements\n";
```

```
29          std::cout << "5. Print the elements\n";
30          std::cout << "6. Exit\n";
31          std::cout << "Enter your choice: ";
32          std::cin >> choice;
33
34          switch (choice) {
35
36          case 1:
37              push();
38              break;
39
40          case 2:
41              pop();
42              break;
43
44          case 3:
45              Top();
46              break;
47
48          case 4:
49              if (isEmpty())
50                  std::cout << "Stack is Empty.\n";
51              else
52                  std::cout << "Stack is NOT Empty.\n";
53              break;
```

```
55              case 5:
56                  print();
57                  break;
58
59              case 6:
60                  std::cout << "Program terminated.\n";
61                  return 0;
62
63              default:
64                  std::cout << "Invalid choice.\n";
65              }
66          }
67
68          return 0;
69      }
70
71      bool isEmpty() {
72          return top == -1;
73      }
74
75      bool isFull() {
76          return top == i - 1;
77      }
```

```
74
75  bool isFull() {
76      return top == i - 1;
77  }
78
79  void push() {
80
81      if (isFull()) {
82          std::cout << "Stack Overflow!\n";
83          return;
84      }
85
86      std::cout << "Enter value to push: ";
87      std::cin >> newData;
88
89      stack[++top] = newData;
90
91      std::cout << newData << " pushed successfully.\n";
92  }
93
94  void Top() {
95
96      if (isEmpty()) {
97          std::cout << "Stack Underflow!\n";
98          return;
99      }
```

```
97          std::cout << "Stack Underflow!\n";
98          return;
99      }
100
101     std::cout << "Top element: " << stack[top] << std::endl;
102 }
103
104 void pop() {
105
106     if (isEmpty()) {
107         std::cout << "Stack Underflow!\n";
108         return;
109     }
110
111     std::cout << "Popped: " << stack[top] << std::endl;
112     top--;
113 }
114
115 void print() {
116
117     if (isEmpty()) {
118         std::cout << "Stack is Empty.\n";
119         return;
120     }
121
122     std::cout << "Stack Elements (Top to Bottom): ";
123
124     for (int j = top; j >= 0; j--) {
```

```
122     std::cout << "Stack Elements (Top to Bottom): ";
123
124     for (int j = top; j >= 0; j--) {
125         std::cout << stack[j] << " ";
126     }
127
128     std::cout << std::endl;
129 }
```

OUTPUT

```
===== STACK OPERATIONS =====
1. PUSH
2. POP
3. TOP
4. Is Empty
5. Print the elements
6. Exit
Enter your choice: 1
Enter value to push: 99
99 pushed successfully.
```

```
===== STACK OPERATIONS =====
1. PUSH
2. POP
3. TOP
4. Is Empty
5. Print the elements
6. Exit
Enter your choice: 1
Enter value to push: 99
99 pushed successfully.
```

```
===== STACK OPERATIONS =====
1. PUSH
2. POP
3. TOP
4. Is Empty
5. Print the elements
6. Exit
Enter your choice: 2
Popped: 25
```

```
===== STACK OPERATIONS =====
1. PUSH
2. POP
3. TOP
4. Is Empty
5. Print the elements
6. Exit
Enter your choice: 2
Popped: 25
```

```
===== STACK OPERATIONS =====
1. PUSH
2. POP
3. TOP
4. Is Empty
5. Print the elements
6. Exit
Enter your choice: 5
Stack Elements (Top to Bottom): 25
```

```
===== STACK OPERATIONS =====
1. PUSH
2. POP
3. TOP
4. Is Empty
5. Print the elements
6. Exit
Enter your choice: 5
Stack is Empty.
```

```
 Stack Empty?:  1
Stack Empty?:  0
The size of the stack: 3
The top of the element is: 15
Removing the elements from the stack:
The top of the element: is: 15
The top of the element: is: 8
The top of the element: is: 3
PS C:\Alvarez CPE21S1\STACK>
```

Observation and Analysis

- In this activity the first thing i notice, is how Push works, push is after i input a number it will retained into stack and after testing out the other functions i secondly test the pop which is if i put anything a number and push it into the stack , and if i pop the first stack it will vanished in the stack, third thing i notice is the Top which is if it checks whats the Top of my stack after testing multiple number that i push in the stack so the main function of top is to check what is the first number in the top, Then lastly i noticed is how can i display or how can i see the stack which is i include another function is the Print the elements that i combine in the stack.

ILO B: Develop C++ code that uses both arrays and linked lists to create a stack

```cpp
#ifndef STACKLIST_LL_H
#define STACK_LIST_H
#include <iostream>

//class node that contains data and the next
template <typename T>
class Node{
    public:
    T data;     //value that we need store
    Node* next; //pointer to the next node
};

//global pointer for the top of the stack
template <typename T>
Node <T>*head = nullptr;

//operation for the stack

//push
template <typename T>
void push(T newData){
    //create a new node and assign the newData
    Node<T>* newNode = new Node<T>;

    newNode->data = newData;
```

```
26      //newnode points to current top, then becomes the new top
27      newNode->next = head<T>;
28      head<T> = newNode;
29  }
30
31  //pop
32  template<typename T>
33  T pop(){
34      //check if the stack is empty;
35      if(head<T> == nullptr){
36          std::cout << "Stack Overflow. "<< std::endl;
37      }
38
39      //save the current top node
40      Node<T>* temp = head<T>;
41      //store its data
42      T temVal = temp->data;
43      //move the head to the next node
44      head<T> = head<T>->next;
45      //free the removed node
46      delete temp;
47      //return the removed value
48      return temVal;
49  }
50
```

```
50
51  //top
52  template<typename T>
53  void Top(){
54      //error catching
55      if(head<T> == nullptr){
56          std::cout<<"Stack undeflow "<<std::endl;
57          return;
58      }
59      //display the Top node
60      std::cout<<"Top of Stack: "<<head<T>->data <<std::endl;
61  }
62
63  //isEmpty
64  template<typename T>
65  bool isEmpty(){
66      return head <T> == nullptr;
67  }
68
69  // displayAll
70  template <typename T>
71  void displayAll() {
72      if (head<T> == nullptr) {
73          std::cout << "Stack is empty." << std::endl;
74          return;
75      }
```

S

```
75      }
76      Node<T>* temp = head<T>;
77      while (temp != nullptr) {
78          std::cout << temp->data << "\n";
79          temp = temp->next;
80      }
81      std::cout << std::endl;
82  }
83
84  #endif
```

Main.cpp

ILOB > main.cpp > ...

```cpp
//Alvarez,Joshua Caleb P.
#include <iostream>
#include "stacklist.h"

int main() {
    std::cout << "------ PUSHING 'c' 'p' 'e' ONTO THE STACK ------ \n";

    // Push elements
    push<char>('c');
    push<char>('p');
    push<char>('e');

    // Display the current top
    std::cout << "Top: " << Top<char>() << std::endl; // Top should be 'e'
    displayAll<char>();
    std::cout << std::endl;

    std::cout << "\n ------ Popping all the elements ------- \n";

    while (!isEmpty<char>()) {
        std::cout << "Popped: " << pop<char>() << std::endl;
    }

    std::cout << "\n ------ Testing the Underflow ------ \n";
    pop<char>(); // Should trigger underflow handling in your stacklist.h

    return 0;
}
```

OUTPUT

```
------ PUSHING 'c' 'p' 'e' ONTO THE STACK ------
Top: e
Stack contents:
Top -> e -> p -> c
```

```
------ POPPING ALL THE ELEMENTS ------
Popped: e
Popped: p
Popped: c
```

```
------ TESTING UNDERFLOW ------
Stack underflow.
```

ILOB TASKS
Tasks:
- **Modify the code given above to include a function that will display all elements in the stack.**
- **Provide a description of each operation provided.**
- **Include your output in the output section of your activity template.**

CODE

Stacks > main.cpp > main()

```cpp
#include <iostream>
#include "stackList.h"

using namespace std;

int main() {

    int choice;
    char value;

    do {

        cout << "\n============================\n";
        cout << "       STACK MENU\n";
        cout << "============================\n";
        cout << "1. Push\n";
        cout << "2. Pop\n";
        cout << "3. Top\n";
        cout << "4. Display All\n";
        cout << "5. Is Empty\n";
        cout << "6. Exit\n";
```

```
22          cout << "============================\n";
23          cout << "Enter your choice: ";
24          cin >> choice;
25
26          switch(choice){
27
28          case 1:
29
30              cout << "Enter character to push: ";
31              cin >> value;
32
33              push<char>(value);
34
35              cout << "'" << value << "' has been pushed.\n";
36              break;
37
38          case 2:
39
40              if(isEmpty<char>())
41                  cout << "Stack is empty.\n";
```

```
42              else
43                  cout << "Popped: " << pop<char>() << endl;
44
45              break;
46
47          case 3:
48
49              if(isEmpty<char>())
50                  cout << "Stack is empty.\n";
51              else
52                  cout << "Top element: " << Top<char>() << endl;
53
54              break;
55
56          case 4:
57
58              cout << "\nStack contents:\n";
59              displayAll<char>();
60
61              break;
```

```
63          case 5:
64
65              if(isEmpty<char>())
66                  cout << "Stack is EMPTY.\n";
67              else
68                  cout << "Stack is NOT EMPTY.\n";
69
70              break;
71
72          case 6:
73
74              cout << "Thank you!\n";
75              break;
76
77          default:
78
79              cout << "Invalid choice.\n";
80
81          }
82
83      } while(choice != 6);
84
85      return 0;
86  }
```

OUTPUT

```
============================
       STACK MENU
============================
1. Push
2. Pop
3. Top
4. Display All
5. Is Empty
6. Exit
============================
Enter your choice: 1
Enter character to push: G
'G' has been pushed.
```

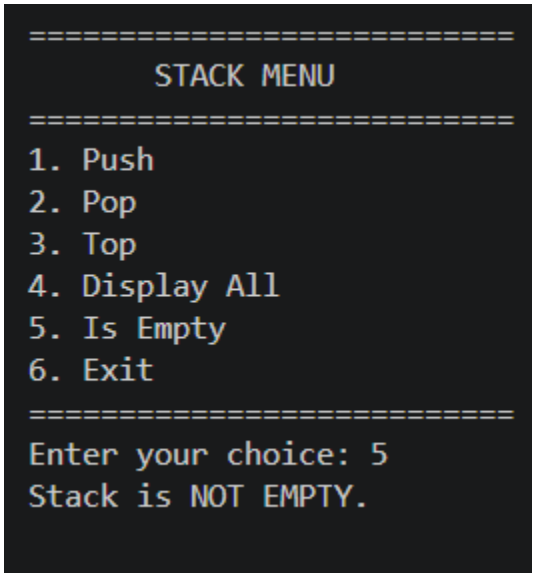
```
============================
       STACK MENU
============================
1. Push
2. Pop
3. Top
4. Display All
5. Is Empty
6. Exit
============================
Enter your choice: 4

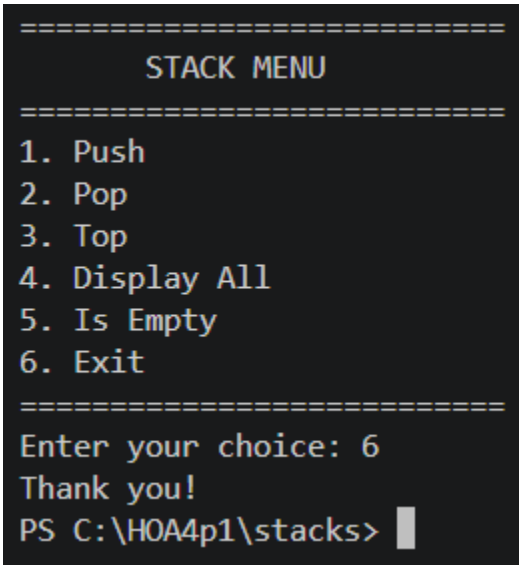
Stack contents:
Top -> C -> J -> C -> F -> G

============================
```

```
============================
       STACK MENU
============================
1. Push
2. Pop
3. Top
4. Display All
5. Is Empty
6. Exit
============================
Enter your choice: 2
Popped: C
```

```
============================
       STACK MENU
============================
1. Push
2. Pop
3. Top
4. Display All
5. Is Empty
6. Exit
============================
Enter your choice: 3
Top element: J
```





Observation and Analysis

Observation

During the activity,I successfully implemented a stack using a linked list and tested its basic operations such as push, pop, top,Display all and isEmpty.The menu that i built allowed the user to interact by inputting the choice by entering any values between 1-6 and as i say that the user interact with the stack by selecting different operations, I also observed that the stack follows the last in, first out **(LIFO)** principle, where the most recently added element is the first one removed.

Analysis

After fixing and modify i firstly notice that its actually run even i first time to use switch case and calling out the function of the different files which is stacklist and as i observed each operation is can be function when the dynamic memory allocation called out in the main cpp file and also i noticed that i debug different type of error that it takes time to process and after this activity i noticed also that terminal and output are not the same, output is just for one time use and its like not mainly for user interaction but the terminal is the opposite which is if u run the code u can input some of the details.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve problems using an implementation of stack:

The following problem definition and algorithm is provided for checking the balancing of symbols. Create an implementation using stacks. Your output must include the following:

a. Stack using Arrays
b. Stack using Linked Lists
c. (Optional) Stack using C++ STL Problem Definition:

Stacks can be used to check whether the given expression has balanced symbols. This algorithm is very useful in compilers. Each time the parser reads one character at a time. If the character is an opening delimiter such as (, {, or [- then it is written to the stack. When a closing delimiter is encountered like), }, or]-the stack is popped. The opening and closing delimiters are then compared. If they match, the parsing of the string continues. If they do not match, the parser indicates that there is an error on the line.

Steps:
1. Create a Stack.
2. While(end of input is not reached)
{ a) If the character read is not a symbol to be balanced, ignore it.
b) If the character is an opening symbol, push it onto the stack.
c) If it is a closing symbol: i) Report an error if the stack is empty. ii) Otherwise, pop the stack.
d) If the symbol popped is not the corresponding opening symbol, report an error.
3. At the end of input, if the stack is not empty: report an error. Self-Checking:

For the following cases, complete the table using the code you created.

CODE

Supmain.cpp

Stacks > Supplementary activity > Supmain.cpp > ...

```cpp
// Alvarez, Joshua Caleb P.
#include <iostream>
#include <string>
#include "symbolStack.h"

bool isOpeningBracket(char symbol)
{
    return symbol == '(' || symbol == '{' || symbol == '[';
}

bool isClosingBracket(char symbol)
{
    return symbol == ')' || symbol == '}' || symbol == ']';
}

bool bracketsMatch(char openBracket, char closeBracket)
{
    return (openBracket == '(' && closeBracket == ')') ||
           (openBracket == '{' && closeBracket == '}') ||
           (openBracket == '[' && closeBracket == ']');
}

bool checkBalancedExpression(const std::string& expression)
{
    // Clear the stack
    while (!isEmpty<char>())
    {
        pop<char>();
    }
```

```
31      // Read every character
32      for (char symbol : expression)
33      {
34          if (isOpeningBracket(symbol))
35          {
36              push<char>(symbol);
37          }
38          else if (isClosingBracket(symbol))
39          {
40              if (isEmpty<char>())
41              {
42                  return false;
43              }
44
45              char topSymbol = pop<char>();
46
47              if (!bracketsMatch(topSymbol, symbol))
48              {
49                  return false;
50              }
51          }
52      }
53
54      return isEmpty<char>();
55  }
```

```cpp
int main()
{
    std::string expression;

    std::cout << "==============================\n";
    std::cout << " Balanced Symbols Checker\n";
    std::cout << "==============================\n\n";

    std::cout << "Enter an expression: ";
    std::getline(std::cin, expression);

    if (checkBalancedExpression(expression))
    {
        std::cout << "\nVALID Expression\n";
    }
    else
    {
        std::cout << "\nINVALID Expression\n";
    }

    return 0;
}
```

CODE

symbolStack.h

```
// Alvarez, Joshua Caleb P.
#ifndef SYMBOLSTACK_H
#define SYMBOLSTACK_H

#include <iostream>

template <typename T>
class Node
{
public:
    T data;
    Node<T>* next;
};

template <typename T>
inline Node<T>* head = nullptr;

template <typename T>
void push(T newData)
{
    Node<T>* newNode = new Node<T>;
    newNode->data = newData;
    newNode->next = head<T>;
    head<T> = newNode;
}

```

```cpp
template <typename T>
T pop()
{
    if (head<T> == nullptr)
    {
        std::cout << "Stack underflow.\n";
        return T();
    }

    Node<T>* temp = head<T>;
    T value = temp->data;
    head<T> = head<T>->next;
    delete temp;

    return value;
}

template <typename T>
bool isEmpty()
{
    return head<T> == nullptr;
}

#endif
```

OUTPUT

```
==============================
 Balanced Symbols Checker
==============================

Enter an expression: (A+B)+(C-D)

VALID Expression
PS C:\HOA4p1\stacks\Supplementary activity>
```

```
==============================
 Balanced Symbols Checker
==============================

Enter an expression: ((A+B)+(C-D)

INVALID Expression
```

```
==============================
 Balanced Symbols Checker
==============================

Enter an expression: ((A+B)+[C-D])

VALID Expression
```

```
==============================
 Balanced Symbols Checker
==============================

Enter an expression: ((A+B]+[C-D]}

INVALID Expression
```

Expression	Valid? (Y/N)	Output (Console Screenshot)	Analysis
(A+B)+(C-D)	Y	VALID Expression	The expression is valid because every opening parenthesis has a corresponding closing parenthesis, and all symbols are correctly matched
((A+B)+(C-D)	N	INVALID Expression	The expression is invalid because one opening parenthesis (does not have matching closing parenthesis, leaving the stack non-empty.
((A+B)+[C-D])	Y	VALID Expression	The expression is valid because the parentheses() and square brackets [ ] are properly nested and matched.
((A+B]+[C-D]}	N	INVALID Expression	The expression is invalid because the opening and closing symbols do not match correctly ((is closed by] and also (mismatched with }).

Summary of Lessons Learned

Through this activity, I learned how the **stack** data structure works using the **Last in, First out (LIFO)** principle. I gained experience implementing the basic stack operations such as **push,pop,top,display, and isEmpty** using a linked list. I also learned how stacks can be applied to real world problems, such as checking whether symbols in an expression are balanced.

Analysis of the Procedure

The procedure involved creating a stack using a linked list and testing each stack operation through an interactive program. After verifying that the stack worked correctly, the same implementation was used to solve the balanced symbols problem. By pushing opening symbols onto the stack and popping them when matching closing symbols were encountered, the program was able to determine whether an expression was valid or invalid. Following the procedure step by step helped ensure that the program functioned correctly.

Analysis of the Supplementary Activity

In this supplementary activity it demonstrated the practical application of stacks in checking balanced symbols. The program successfully identified whether expressions contained properly matched parentheses, brackets , and braces. This activity showed how the stacked and LIFO behaviour makes it suitable for validating nested symbols. It also enhanced my understanding of stack operations and their importance in programming applications such as syntax checking and expression validation.

Concluding Statement / Feedback

I think I did well in this activity because I was able to finish the required stack operations and make the balanced symbols checker work correctly. At first, I encountered several errors while coding and compiling the program, but I was able to fix them by checking my code and testing it multiple times. This activity helped me understand how stacks work and how they can be used in solving problems. I still need to improve my C++ programming skills, especially in debugging errors, organizing my code, and understanding functions better so I can complete similar activities more confidently in the future.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Chatgpt and quillbot**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction, summarize the concepts, code assistance ,explanation of concepts and also code idea generation**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I use ai just to help me debug errors and ai help me trough suggestions he make but i also conceptualize my idea**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.